FUSION LEAP AI GROWTH

Guide de recette · 12 novembre 2026

Tests fonctionnels Phases 2 et 3

Projets, leads et deals, capture, import, assignation, synchronisation CRM, intégrations, webhooks e-mail, conversations et envoi sous contrôles : de J16 à J32.

107	~5 h	2
cas de test	durée estimée	parties : interface, technique

Comment utiliser ce guide

1. Préparer l'environnement et charger les données (section 0).
2. Dérouler les cas dans l'ordre : chaque ligne indique les étapes et le **résultat attendu**.
3. Noter **OK** ou **KO** dans la **fiche de résultats** (dernières pages).
4. Pour chaque KO : identifiant du cas, ce qui a été fait et vu, capture d'écran.

Sommaire

0. Préparation (une seule fois)

PARTIE 1 — INTERFACE

A. Projets et produits (J16, J17)
B. Leads (J24)
C. Deals (J24)
D. Intégrations (J28)

PARTIE 2 — TECHNIQUE

T0. Suites de tests automatiques
T1. Capture publique de leads (J21)
T2. Assignation automatique (J22)
T3. Import CSV de leads (J23)
T4. Synchronisation CRM par fichier (J25)
T5. Intégrations, secrets et secret de webhook (J28, J32)
T6. Webhooks e-mail : statuts et réponses (J29, J32)
T7. Conversations et droits de lecture (J30)
T8. Envoi sous contrôles (J31)
T9. Isolation et droits (vérifications rapides)
T10. Nettoyage
Ce que ce guide ne couvre pas (à connaître)
Fiche d'anomalie

Tests fonctionnels — Phases 2 et 3 (J16 à J32)

Guide pour la personne qui teste (recette). Il couvre ce qui a été construit après la Phase 1 : projets et produits, leads et deals, capture de leads, import CSV, assignation automatique, synchronisation CRM, intégrations, webhooks e-mail, conversations et envoi sous contrôles. La Phase 1 (connexion, membres, rôles, plans, thème) est dans `phase1-functional-tests.md`.

- **Durée estimée** : 3 heures pour la partie Interface, 2 heures pour la partie Technique.
- **Règle d'or** : pour chaque cas, noter **OK** ou **KO** dans la fiche de résultats (dernières pages du PDF). En cas de **KO**, noter l'identifiant du cas, ce que vous avez fait, ce que vous avez vu, et joindre une capture d'écran.
- **Données** : uniquement des données de test (adresses en `.test`). Ne jamais utiliser de vraies adresses de clients : **aucun e-mail réel n'est envoyé** par ces tests.
- **Deux parties** : la partie **Interface** se fait à la souris dans l'application. La partie **Technique** concerne des fonctions qui n'ont **pas d'écran** (capture publique, import, webhooks, envoi) : elle se fait avec des commandes à copier-coller, dont le résultat attendu est indiqué.

---

0. Préparation (une seule fois)

0.1 Installer et démarrer

1. Suivre `docs/runbooks/setup-new-machine.md` (Docker Desktop, Node 22.12 ou plus, pnpm, `.env.local` avec une `SECRETS_MASTER_KEY` locale : sans elle, les intégrations répondent « chiffrement non configuré »).
2. **Utiliser Git Bash** (installé avec Git) pour toutes les commandes de ce guide. Les guillemets des commandes ne fonctionnent pas dans PowerShell.
3. Démarrer, dans trois terminaux :

```
pnpm db:start && pnpm db:reset      # base locale remise à l'état de départ
pnpm dev                            # l'application : http://localhost:3000
pnpm worker:dev                     # le worker : traite les tâches de fond (import, assignation, webhooks...)
```

   Si `pnpm dev` ne démarre pas, construire puis lancer l'application en mode production : `pnpm build` puis, dans `apps/web`, `node_modules/.bin/next start -p 3000` (les variables de `.env.local` doivent être exportées dans ce terminal).

4. Ouvrir **un seul** navigateur à l'adresse `http://localhost:3000` (ne jamais mélanger `localhost` et `127.0.0.1` : les sessions sont séparées).

0.2 Comptes de test

Mot de passe de développement (le même pour tous) : constante `dev_password` au début de `supabase/seed.sql`.

ADRESSE	RÔLE	ENTREPRISE(S)
`shared@seed.test`	Propriétaire (Alpha), Commercial (Beta), **administrateur de la plateforme**	Alpha, Beta
`alpha-admin@seed.test`	Administrateur	Alpha
`alpha-marketing@seed.test`	Responsable marketing	Alpha
`alpha-sales@seed.test`	Commercial	Alpha
`alpha-analyst@seed.test`	Analyste	Alpha
`beta-owner@seed.test`	Propriétaire	Beta

Pour tester deux comptes en même temps, utiliser une fenêtre de navigation privée pour le second.

0.3 Charger les données de démonstration

La base de départ n'a ni leads ni deals. Charger le jeu de démonstration (8 contacts, 8 leads dont 2 non assignés, 4 deals) **une seule fois** après `pnpm db:reset` :

```
docker exec -i supabase_db_fusion-leap psql -U postgres -v ON_ERROR_STOP=1 < docs/testing/fixtures/phase2-demo-data.sql
```

Résultat attendu, en dernières lignes : `contacts: 8`, `leads: 8`, `deals: 4`. Un second chargement est refusé (doublons d'e-mail) : refaire `pnpm db:reset` pour repartir de zéro.

0.4 L'outil `pnpm test:call` (partie Technique)

`pnpm test:call <compte> <METHODE> <chemin> [corps-json] [--org alpha|beta] [--file fichier]` appelle l'API **en tant que** un compte de test (même session que l'interface). Exemple : `pnpm test:call shared GET /api/integrations`. Le compte est la partie avant `@seed.test`. Si l'application n'est pas sur `http://localhost:3000`, définir `E2E_BASE_URL`.

0.5 Deux raccourcis pour les vérifications en base (partie Technique)

Dans **chaque** terminal Git Bash utilisé pour la partie Technique, coller :

```
q()  { docker exec supabase_db_fusion-leap psql -U postgres -c "$1"; }          # affiche un tableau
qa() { docker exec supabase_db_fusion-leap psql -U postgres -t -A -c "$1"; }    # affiche la valeur brute
```

Ensuite `q "select …"` interroge la base locale.

0.6 À savoir (ce ne sont pas des anomalies)

- Les étapes du pipeline d'Alpha s'affichent **en anglais** (Unassigned, New, Qualified, Proposal, Won, Lost) : Alpha est configurée en anglais.
- La colonne **Responsable** montre « Moi » pour votre propre compte et **« Membre 00000x »** pour les autres (identifiant partiel, pas encore le nom).
- Les messages d'erreur renvoyés par l'API (partie Technique) sont **en anglais**.
- Dans les commandes `curl`, **n'écrire aucun accent** dans les textes JSON : la console Windows ne les envoie pas en UTF-8 et l'application refuse un texte invalide (réponse 400, voulue).
- Rien ne part vers l'extérieur : l'envoi réel d'e-mails n'est pas activé (il manque un compte d'essai chez le fournisseur).

PARTIE 1 — INTERFACE

A. Projets et produits (J16, J17)

Menu **Projets**. Compte : `shared@seed.test` (propriétaire d'Alpha) sauf mention contraire.

ID	ÉTAPES	RÉSULTAT ATTENDU
A1	Ouvrir **Projets**.	Tableau avec **Site vitrine Alpha** (Vente de sites web, Actif) et **Application Salla Alpha** (Application Salla, Actif). Bouton **Nouveau projet** et filtres **Actifs** / **Archivés**.
A2	Changer d'organisation (**Changer**) pour **Beta**.	Seulement **Boutique en ligne Beta** et **Thème Salla Beta**, jamais les projets d'Alpha. Le message « Votre rôle permet de consulter les projets, pas de les modifier. » s'affiche (vous êtes commercial chez Beta). Revenir dans Alpha.
A3	**Nouveau projet** : type **Vente de sites web**, nom `Projet test 1`, dans Offres et formules ajouter une ligne `Pack test`, puis **Créer le projet**.	Notification verte « Projet créé. » Le projet apparaît dans la liste, statut **Actif**.
A4	Ouvrir **Projet test 1**.	Les réglages saisis s'affichent. Le type **ne peut pas être modifié** : « Le type d'un projet ne peut pas être modifié. »
A5	**Nouveau projet** : laisser le nom vide, valider. Puis un nom de 121 caractères. Puis, dans une page d'atterrissage, une adresse `http://exemple.ma` (sans **s**).	Refus à chaque fois (nom obligatoire, 120 caractères au maximum, adresse en https obligatoire) avec le message « Certaines valeurs sont invalides… ». Aucun projet n'est créé.
A6	Dans **Projet test 1**, modifier une valeur (ex. ajouter `Pack premium`) et **Enregistrer**.	Notification verte « Projet enregistré. » La valeur est conservée après rechargement de la page.

ID	ÉTAPES	RÉSULTAT ATTENDU
A7	Section **Produits et offres** : **Ajouter un produit** : type `pack`, offre `Offre de test`, prix indicatif `à partir de 1 000 MAD`, **Ajouter**.	Notification « Produit ajouté. » Le produit apparaît. **Modifier** (changer l'offre) puis **Enregistrer** : « Produit enregistré. »
A8	Ouvrir **Modifier** sur le produit, section **Fichiers** : envoyer une image PNG de moins de 5 Mo, puis un PDF.	« Fichier envoyé. » à chaque fois ; les fichiers sont listés.
A9	Même endroit : envoyer un fichier `.txt`, puis un fichier de plus de 5 Mo, puis un fichier vide.	Refus : « Type de fichier non accepté : PNG, JPEG, WebP ou PDF seulement. », puis « Ce fichier dépasse 5 Mo. », puis « Ce fichier est vide. »
A10	Cliquer **Ouvrir** (ou **Télécharger**) sur un fichier, puis **Retirer**.	Le fichier s'ouvre (lien temporaire). Après **Retirer** : « Fichier retiré. »
A11	Sur **Projet test 1** : **Archiver**.	Confirmation « Archiver ce projet ? Il deviendra lecture seule et ne comptera plus dans la limite de votre plan. » Après **Confirmer** : « Projet archivé. » Il disparaît de **Actifs** et apparaît dans **Archivés**.
A12	Ouvrir le projet archivé.	Bandeau « Ce projet est archivé : il est en lecture seule. Réactivez-le pour le modifier. » Aucune modification possible. **Réactiver** : « Projet réactivé. »
A13	Avec `alpha-sales@seed.test`, `alpha-marketing@seed.test` puis `alpha-analyst@seed.test` : ouvrir **Projets**. Puis taper directement `http://localhost:3000/fr/projects/new` (commercial ou marketing).	La liste est visible, avec « Votre rôle permet de consulter les projets, pas de les modifier. » et **sans** bouton Nouveau projet. L'adresse `/projects/new` ramène à la liste.
A14	`shared@seed.test` → **Administration** → ouvrir **Alpha** → formule **Essai**, motif `test quota`, **Enregistrer l'abonnement**. Puis **Projets** → **Nouveau projet** → remplir → **Créer le projet**.	Refus : « La limite de projets de votre plan est atteinte. Contactez l'administrateur de la plateforme. » (l'essai autorise 1 projet, Alpha en a déjà).
A15	Rétablir la formule **Croissance** dans **Administration**, puis réessayer la création.	La création réussit de nouveau.
A16	**Nouveau projet** : remplir, puis cliquer **deux fois rapidement** sur **Créer le projet**.	**Un seul** projet est créé (pas de doublon).

B. Leads (J24)

Menu **Leads**. À ce stade, seules les données de démonstration existent (8 leads). Compte : `shared@seed.test`.

ID	ÉTAPES	RÉSULTAT ATTENDU
B1	Ouvrir **Leads**.	« Résultats : 8 ». Colonnes **Contact, E-mail, Téléphone, Source, Étape, Responsable, Créé le**. Contacts : Nadia Benali, Karim Alaoui, Salma Idrissi, Youssef Tazi, Imane Chraibi, Omar Berrada, Lina Fassi, Hamid Zniber.
B2	**Recherche** `nadia` puis **Appliquer**. Puis `xyz`.	« Résultats : 1 » (Nadia Benali). Puis « Aucun lead ne correspond à ces filtres. »
B3	**Réinitialiser**. Choisir **Étape** = `Unassigned` puis **Appliquer**.	« Résultats : 2 » : **Lina Fassi** et **Hamid Zniber**, Responsable **Non assigné**.
B4	**Réinitialiser**. Cliquer le bouton **Non assignés**. Cliquer à nouveau (« Retirer le filtre « non assignés » »).	Le même résultat que B3 (2 leads), puis retour à 8.
B5	**Source** = **Import CSV**, puis **Formulaire web**, puis **Saisie manuelle**.	2 leads (Salma Idrissi, Omar Berrada), puis 4 (Nadia, Karim, Imane, Hamid), puis 2 (Youssef, Lina).
B6	**Responsable** = **Membre 000004** (le commercial).	« Résultats : 6 ».
B7	**Réinitialiser**, puis **Tri** = **Nom (A–Z)**.	Le premier est **Hamid Zniber**, puis Imane Chraibi, Karim Alaoui, Lina Fassi…
B8	**Projet** = **Application Salla Alpha**, **Appliquer**.	Aucun lead (tous les leads de démonstration sont sur le Site vitrine).
B9	**Réinitialiser**, puis **Exporter en CSV**. Ouvrir le fichier.	Un CSV avec une ligne par lead (en-têtes `lead_id, created_at, name, email, phone, source, stage, assignee_id, project_id`). La mention « l'export respecte les filtres actifs (10 000 lignes au plus) et est enregistré dans le journal d'audit » est visible.

ID	ÉTAPES	RÉSULTAT ATTENDU
B10	Vérifier l'audit de l'export : dans un terminal, `q "select action, after from public.audit_logs where action = 'export.created' order by created_at desc limit 1"`.	Une ligne `export.created` avec `{"kind": "leads", "rows": 8, "filters": []}` : le **nombre de lignes** et les **noms** des filtres, jamais leurs valeurs, jamais de coordonnées.
B11	Avec `alpha-sales@seed.test` : ouvrir **Leads**.	Message « Vous voyez uniquement les leads qui vous sont assignés. » « Résultats : 6 », Responsable **Moi**. **Lina Fassi** et **Hamid Zniber** (non assignés) n'apparaissent pas. **Pas** de bouton **Exporter en CSV**.
B12	Avec `alpha-marketing@seed.test` : ouvrir **Leads**. Avec `alpha-analyst@seed.test` : essayer d'ouvrir `/fr/leads`.	Le marketing voit les **8** leads. L'analyste est **renvoyé vers le Tableau de bord**.

C. Deals (J24)

Menu **Deals**. Compte : `shared@seed.test`.

ID	ÉTAPES	RÉSULTAT ATTENDU
C1	Ouvrir **Deals**.	Colonnes **New 1**, **Qualified 1**, **Proposal 1**, **Won 1**, **Lost 0** (« Aucun deal »). Cartes : **Nadia Benali 2 000,00 MAD** (New), **Karim Alaoui 6 500,00 MAD** (Qualified), **Salma Idrissi 12 000,00 MAD** (Proposal), **Youssef Tazi 3 200,00 MAD** (Won).
C2	Sur la carte de Nadia, menu **Déplacer vers…** → **Qualified**.	Notification « Deal déplacé vers « Qualified ». » La colonne New passe à 0 et Qualified à 2. Après rechargement, le déplacement est conservé.
C3	**Glisser-déposer** la carte de Salma vers la colonne **Won**.	Le deal passe en Won (Won : 2, Proposal : 0), même notification.
C4	Déplacer le deal de Youssef (Won) vers **New** (réouverture), puis le remettre en **Won**.	Les déplacements fonctionnent dans les deux sens ; les compteurs sont toujours justes.
C5	**Projet** = **Application Salla Alpha**, **Filtrer**. Puis **Tous les projets**.	Aucun deal pour ce projet (colonnes vides), puis tous les deals reviennent.
C6	Avec `alpha-sales@seed.test` : ouvrir **Deals**, déplacer un deal.	Le commercial voit les **4 deals** (ceux de ses leads) et peut les déplacer.
C7	Avec `alpha-marketing@seed.test` : ouvrir **Deals**.	Les deals sont visibles avec le message « Votre rôle permet de consulter les deals, pas de les déplacer. » (aucun déplacement possible).
C8	Avec `alpha-analyst@seed.test` : essayer d'ouvrir `/fr/deals`.	Renvoyé vers le **Tableau de bord**.

D. Intégrations (J28)

Menu **Intégrations** (propriétaire et administrateur seulement). On crée d'abord une intégration de démonstration, car l'écran ne permet pas encore d'en ajouter :

```
pnpm test:call shared POST /api/integrations '{"provider":"email","external_account_id":"demo-ui-mail","scopes":
["send","receive"],"secret":"jeton-de-test-0123456789-abcdef"}'
```

Résultat attendu : `HTTP 201` et un `id`.

ID	ÉTAPES	RÉSULTAT ATTENDU
D1	`shared@seed.test` → **Intégrations**.	**E-mail (demo-ui-mail)**, État **En attente de validation**, Portée `send receive` , « Jeton enregistré et chiffré », « Le jeton ne peut pas être consulté, même par un administrateur : vous pouvez seulement le remplacer. », « Aucun canal pour cette intégration. »
D2	Vérifier que le jeton n'apparaît nulle part : clic droit → **Afficher le code source** de la page, chercher `jeton-de-test`. Puis `pnpm test:call shared GET /api/integrations`.	Le texte `jeton-de-test` est **absent** de la page et de la réponse de l'API (qui ne montre que `has_secret: true`).

ID	ÉTAPES	RÉSULTAT ATTENDU
D3	Section **Remplacer le jeton** : saisir `court`, **Remplacer**.	Refus : « Le jeton est invalide (8 à 4096 caractères, sans caractère de contrôle). »
D4	Saisir `nouveau-jeton-0123456789`, **Remplacer**.	Notification « Jeton remplacé. » Le champ est vidé ; le nouveau jeton **n'est jamais affiché**.
D5	**Déconnecter**.	Confirmation « Déconnecter cette intégration supprime son jeton et met ses canaux en pause. Cette action est irréversible. » Après **Oui, déconnecter** : « Intégration déconnectée. » État **Déconnectée**, « Aucun jeton enregistré ».
D6	Saisir un nouveau jeton et utiliser **Reconnecter**.	L'intégration repasse à **En attente de validation**, avec un jeton enregistré.
D7	Avec `alpha-admin@seed.test` : ouvrir **Intégrations**. Avec `alpha-marketing`, `alpha-sales` et `alpha-analyst` : essayer d'ouvrir `/fr/integrations`.	L'administrateur voit la page. Les trois autres sont **renvoyés vers le Tableau de bord**.
D8	Avec `beta-owner@seed.test` : ouvrir **Intégrations**.	« Aucune intégration pour l'instant. » : jamais celles d'Alpha.

PARTIE 2 — TECHNIQUE

Ces fonctions n'ont **pas d'écran**. Les commandes se lancent dans Git Bash, à la racine du projet, avec l'application **et le worker** démarrés (§ 0.1). Faire les sections **dans l'ordre** : chacune prépare la suivante.

T0. Suites de tests automatiques

Elles prouvent l'ensemble du code, y compris ce qui ne se voit pas. Toutes doivent être **vertes** (aucun échec).

ID	COMMANDE	RÉSULTAT ATTENDU (AU 12 NOVEMBRE 2026)
T0.1	`pnpm lint && pnpm typecheck`	Aucune erreur.
T0.2	`pnpm test`	Tests unitaires : domaine **654**, adaptateurs **106**, permissions **394**, worker **129**, application web **2 523**, tous verts.
T0.3	`pnpm test:db`	Base de données (pgTAP) : **2 928** tests, 27 fichiers, `Result: PASS`.
T0.4	`pnpm test:rls`	Isolation entre entreprises par la vraie API : **485** tests, 24 fichiers.
T0.5	`pnpm check:leaks`	« Aucune fuite détectée » : aucun secret dans le code envoyé au navigateur.

Les nombres augmentent au fil du projet : ce qui compte est **zéro échec**.

T1. Capture publique de leads (J21)

Un formulaire de site web envoie un lead avec une **clé de formulaire**. Créer une clé pour le projet « Site vitrine Alpha » :

```
docker exec -i supabase_db_fusion-leap psql -U postgres -t -A < docs/testing/fixtures/create-form-key.sql
```

Copier la ligne `CLE=flk_…` (affichée **une seule fois**) dans une variable : `KEY=flk_…`. Puis :

```
B='{"name":"Rania Sebti","email":"rania@demo.test","phone":"06 12 34 56 78","default_country":"MA","language":"fr","consent":
{"channel":"email","purpose":"marketing","text":"J accepte de recevoir des
offres","version":"v1"},"page_url":"https://demo.test/contact","website":""}'
H="-H x-form-key:$KEY -H content-type:application/json -H x-forwarded-for:203.0.113.60"
curl -s -w ' -> %{http_code}\n' $H -H "origin: https://demo.test" -d "$B" http://localhost:3000/api/leads
```

ID	TEST	RÉSULTAT ATTENDU
T1.1	La commande ci-dessus (origine autorisée).	`{"status":"received"} -> 202`.
T1.2	La **même commande** une seconde fois (doublon).	Exactement la même réponse `202` (rien n'indique « déjà connu »).
T1.3	Remplacer l'origine par `-H "origin: https://pirate.test"`.	`403` : une origine non autorisée est refusée.
T1.4	Remplacer la clé par `flk_` suivi de 64 zéros.	`403` : une clé inventée est refusée.
T1.5	Mettre `"website":"robot"` dans le corps (champ piège rempli).	`202` (même réponse), mais **rien n'est écrit**.
T1.6	Vérifier : `q "select count(*) from public.contacts where email_normalized = 'rania@demo.test'"`	`1` : un seul contact malgré les doublons.
T1.7	Ouvrir **Leads** (`shared@seed.test`).	« Résultats : 9 ». **Rania Sebti**, Source **Formulaire web**, Étape **Unassigned**, Responsable **Non assigné**.
T1.8	Vérifier les alertes : `q "select count(*) from public.notifications"`	`2` : le propriétaire et l'administrateur sont alertés d'un lead non assigné.

T2. Assignation automatique (J22)

Créer la règle « chacun son tour » entre le commercial et l'administrateur d'Alpha :

```
docker exec -i supabase_db_fusion-leap psql -U postgres -t -A < docs/testing/fixtures/create-form-key.sql
```

(nouvelle clé : la copier dans `KEY`), puis

```
docker exec -i supabase_db_fusion-leap psql -U postgres -t -A < docs/testing/fixtures/assignment-rule.sql
```

Résultat attendu : `REGLE=<identifiant>`. Puis envoyer 3 leads (le worker doit tourner) :

```
for n in 1 2 3; do curl -s -o /dev/null -w "lead $n -> %{http_code}\n" -H "x-form-key: $KEY" -H content-type:application/json -H x-forwarded-for:203.0.113.61 -H "origin: https://demo.test" -d "{\"name\":\"Auto Test $n\",\"email\":\"auto$n@demo.test\",\"consent\":{\"channel\":\"email\",\"purpose\":\"marketing\",\"text\":\"J accepte\",\"version\":\"v1\"},\"website\":\"\"}" http://localhost:3000/api/leads; done
```

ID	TEST	RÉSULTAT ATTENDU
T2.1	Les 3 envois.	`202` pour chacun.
T2.2	Après 10 secondes : `q "select c.email_normalized, u.email as responsable from public.leads l join public.contacts c on c.organization_id=l.organization_id and c.id=l.contact_id left join auth.users u on u.id=l.assignee_id where c.email_normalized like 'auto_@demo.test' order by c.created_at"`	**auto1 → alpha-sales**, **auto2 → alpha-admin**, **auto3 → alpha-sales** : chacun son tour.
T2.3	**Leads** avec `alpha-sales@seed.test`.	Le commercial voit ses 2 nouveaux leads (Auto Test 1 et 3), pas Auto Test 2.

T3. Import CSV de leads (J23)

L'import se fait en 3 étapes : envoi du fichier, aperçu (qui n'écrit rien), confirmation. Le fichier d'essai contient 100 lignes dont des erreurs volontaires.

```
P=a1000000-0000-4000-8000-000000000001
pnpm test:call shared POST "/api/imports?project_id=$P" --file tests/fixtures/leads-100.csv
```

Copier le `batch_id` obtenu : `BATCH=…`. Puis l'aperçu :

```
M='{"name":0,"email":1,"phone":2,"language":3,"country":4}'
pnpm test:call shared POST /api/imports/preview "{\"batch_id\":\"$BATCH\",\"mapping\":$M,\"default_country\":\"MA\"}"
```

ID	TEST	RÉSULTAT ATTENDU
T3.1	Envoi du fichier.	`HTTP 201`, `row_count: 100`, colonnes `Nom, Email, Téléphone, Langue, Pays`.
T3.2	Aperçu.	`HTTP 200` : `rowCount 100`, **`valid 82`**, **`newContacts 67`**, `existingContacts 0`, **`duplicatesInFile 15`**, **`errors 18`** (`no_identifier 5`, `invalid_email 5`, `invalid_phone 5`, `invalid_language 3`). **Rien n'est encore créé** : Leads affiche toujours le même nombre.
T3.3	Confirmation : `pnpm test:call shared POST /api/imports/confirm "{\"batch_id\":\"$BATCH\",\"mapping\":$M,\"default_country\":\"MA\"}"`	`HTTP 202`, `"status": "queued"`. L'import ne se fait **pas** dans la requête : c'est le worker qui travaille.
T3.4	Après 15 secondes : `pnpm test:call shared GET "/api/imports?id=$BATCH"`	`"status": "done"`, **`created_count 67`**, **`matched_count 15`**, **`error_count 18`**.
T3.5	`pnpm test:call shared GET "/api/imports/report?batch_id=$BATCH"`	Un CSV `row,outcome,reason,lead_created` : une ligne par enregistrement (`created`, `matched`, `error` avec le motif). **Aucun nom, e-mail ou téléphone** dans le rapport.
T3.6	**Leads** (`shared@seed.test`).	« Résultats : 79 », « Page 1 sur 4 » (25 par page). Les leads importés ont la Source **Import CSV**. Utiliser **Page suivante** : les pages 2, 3 et 4 s'affichent.
T3.7	Avec `alpha-sales`, tenter l'import : `pnpm test:call alpha-sales POST "/api/imports?project_id=$P" --file tests/fixtures/leads-100.csv`	`HTTP 403` : seuls le propriétaire et l'administrateur importent.

T4. Synchronisation CRM par fichier (J25)

Le CRM interne est la source de vérité ; un fichier CSV joue le rôle d'un CRM externe. Dans le terminal de T3 (la variable `P` y est définie), suivre les 4 étapes (le fichier est `docs/testing/fixtures/crm-sync-sample.csv`, 4 lignes dont **Nadia**, qui existe déjà) :

```
pnpm test:call shared POST /api/crm/integrations '{"provider":"csv","external_account_id":"demo-csv"}'
```

Copier l'`id` : `INT=…`. Puis le mapping des champs :

```
pnpm test:call shared POST /api/crm/mappings "{\"integration_id\":\"$INT\",\"mappings\":
[{\"entity\":\"contact\",\"internal_field\":\"external_id\",\"external_field\":\"id\",\"source_of_truth\":\"external\",\"direction\":\"pull\"}
,{\"entity\":\"contact\",\"internal_field\":\"email\",\"external_field\":\"email\",\"source_of_truth\":\"internal\",\"direction\":\"pull\"},
{\"entity\":\"contact\",\"internal_field\":\"name\",\"external_field\":\"nom\",\"source_of_truth\":\"external\",\"direction\":\"pull\"},
{\"entity\":\"contact\",\"internal_field\":\"language\",\"external_field\":\"langue\",\"source_of_truth\":\"internal\",\"direction\":\"pull\"}
,{\"entity\":\"contact\",\"internal_field\":\"country\",\"external_field\":\"pays\",\"source_of_truth\":\"internal\",\"direction\":\"pull\"},
{\"entity\":\"contact\",\"internal_field\":\"external_updated_at\",\"external_field\":\"modifie_le\",\"source_of_truth\":\"external\",\"direct
ion\":\"pull\"}]}"
pnpm test:call shared POST "/api/imports?project_id=$P" --file docs/testing/fixtures/crm-sync-sample.csv
```

Copier le `batch_id` : `B2=…`. Puis lancer la synchro : `pnpm test:call shared POST /api/crm/sync "{\"integration_id\":\"$INT\",\"batch_id\":\"$B2\"}"`.

ID	TEST	RÉSULTAT ATTENDU
T4.1	Création de l'intégration, puis du mapping.	`HTTP 201` puis `HTTP 200` `{"saved": 6}`.
T4.2	Lancement de la synchro.	`HTTP 202`, `"status": "queued"`.
T4.3	Après 15 secondes : `q "select email_normalized, name, external_id from public.contacts where email_normalized like 'sync_@demo.test' or email_normalized = 'nadia@demo.test' order by 1"`	4 contacts : `nadia@demo.test` (nom devenu **Nadia B. (CRM)**, identifiant externe `crm-003` : le contact existant est **rattaché**, pas dupliqué), `sync1`, `sync2` et `sync4@demo.test` (créés).

ID	TEST	RÉSULTAT ATTENDU
T4.4	**CRM** (`shared@seed.test`).	**Fichier CSV (demo-csv)**, Dernière synchronisation à la date du jour, État **Réussie**, tableau **Correspondance des champs** (Champ interne, Champ du fournisseur, Source de vérité, Sens).
T4.5	**Deals** : la carte de Nadia.	Le nom affiché est maintenant **Nadia B. (CRM)** (la source de vérité du nom est le fournisseur).
T4.6	**CRM** avec `alpha-sales` , `alpha-marketing` , `alpha-analyst` .	Renvoyés vers le Tableau de bord.

T5. Intégrations, secrets et secret de webhook (J28, J32)

Créer l'intégration e-mail utilisée par la suite :

```
pnpm test:call shared POST /api/integrations '{"provider":"email","external_account_id":"demo-mail-guide","scopes":
["send","receive"],"secret":"jeton-de-test-0123456789-abcdef"}'
```

Copier l'`id` : `EID=…`. Le **secret de webhook** est le mot de passe que le fournisseur d'e-mail enverra avec chaque événement ; il est **différent** du jeton d'envoi.

ID	TEST	RÉSULTAT ATTENDU
T5.1	`pnpm test:call alpha-sales POST /api/integrations/webhook-secret "{\"integration_id\":\"$EID\",\"secret\":\"mot-de-passe-de-webhook-0123456789-abcdef\"}"`	`HTTP 403` , code `forbidden` : le commercial ne peut pas.
T5.2	Même commande avec `shared` et le secret `court` .	`HTTP 400` , `validation_failed` (32 caractères au minimum).
T5.3	Même commande avec `shared` et le secret **`jeton-de-test-0123456789-abcdef`** (le même que le jeton d'envoi).	`HTTP 400` : le secret de webhook ne peut pas être le jeton d'envoi.
T5.4	Même commande avec `shared` et `mot-de-passe-de-webhook-0123456789-abcdef` .	`HTTP 200` `{"replaced": true}` : le secret n'est **jamais** renvoyé.
T5.5	`qa "select has_secret, has_webhook_secret, status from public.integrations where id = '$EID'"`	`t|t|pending` .
T5.6	`qa "select count(*) from private.secrets where ciphertext like '%mot-de-passe%' or ciphertext like '%jeton-de-test%'"`	`0` : les secrets sont **chiffrés** en base, jamais en clair.
T5.7	`pnpm test:call beta-owner POST /api/integrations/webhook-secret "{\"integration_id\":\"$EID\",\"secret\":\"mot-de-passe-de-webhook-0123456789-abcdef\"}" --org beta`	`HTTP 404` : Beta ne peut pas toucher l'intégration d'Alpha.

T6. Webhooks e-mail : statuts et réponses (J29, J32)

Le fournisseur (Postmark) envoie des événements à `/api/webhooks/email` avec un **Basic Auth** : utilisateur = identifiant du compte (`demo-mail-guide`), mot de passe = secret de webhook. D'abord, préparer 3 e-mails « déjà acceptés par le fournisseur » (aucun envoi réel) et activer le canal :

```
docker exec -i supabase_db_fusion-leap psql -U postgres -v ON_ERROR_STOP=1 < docs/testing/fixtures/phase3-email-demo.sql
```

Résultat attendu : un tableau avec `karim@demo.test` , `nadia@demo.test` , `youssef@demo.test` , tous **accepted**. Puis définir :

```
U='-u demo-mail-guide:mot-de-passe-de-webhook-0123456789-abcdef'
H='-H x-forwarded-for:203.0.113.50 -H content-type:application/json'
URL=http://localhost:3000/api/webhooks/email
D='{"RecordType":"Delivery","MessageID":"pm-guide-nadia","Recipient":"nadia@demo.test","DeliveredAt":"2026-11-12T20:00:00Z","Metadata":{}}'
BOUNCE='{"RecordType":"Bounce","ID":8100001,"Type":"HardBounce","TypeCode":1,"MessageID":"pm-guide-
karim","Email":"karim@demo.test","BouncedAt":"2026-11-12T20:01:00Z","Metadata":{}}'
COMPLAINT='{"RecordType":"SpamComplaint","ID":8100002,"TypeCode":100001,"MessageID":"pm-guide-
youssef","Email":"youssef@demo.test","BouncedAt":"2026-11-12T20:02:00Z","Metadata":{}}'
REPLY='{"From":"nadia@demo.test","FromFull":{"Email":"nadia@demo.test","Name":"Nadia"},"ToFull":
[{"Email":"ventes@demo.test"}],"OriginalRecipient":"ventes@demo.test","Subject":"Re: offre","MessageID":"in-guide-1","TextBody":"Merci, je
suis interessee.","Headers":[{"Name":"Received-SPF","Value":"Pass (ok) identity=mailfrom; envelope-from=nadia@demo.test"}]}'
UNKNOWN='{"From":"inconnu@exemple.test","FromFull":
{"Email":"inconnu@exemple.test"},"OriginalRecipient":"ventes@demo.test","Subject":"Bonjour","MessageID":"in-guide-2","TextBody":"Je decouvre
votre offre.","Headers":[{"Name":"Received-SPF","Value":"Pass identity=mailfrom; envelope-from=inconnu@exemple.test"}]}'
SPOOF='{"From":"karim@demo.test","FromFull":
{"Email":"karim@demo.test"},"OriginalRecipient":"ventes@demo.test","Subject":"Faux","MessageID":"in-guide-3","TextBody":"Ceci est une
usurpation.","Headers":[{"Name":"Received-SPF","Value":"Pass identity=mailfrom; envelope-from=pirate@attacker.test"}]}'
```

Chaque événement s'envoie avec la même commande : `curl -s -w ' -> %{http_code}\n' $U $H -d "$CORPS" $URL`.

ID	TEST	RÉSULTAT ATTENDU
T6.1	`curl -s -o /dev/null -w '%{http_code}\n' -u demo-mail-guide:mauvais $H -d "$D" $URL`	`401` (mauvais mot de passe).
T6.2	La même commande avec `-u inconnu:mot-de-passe-de-webhook-0123456789-abcdef`.	`401` : **la même réponse** (on ne peut pas deviner qu'un compte existe).
T6.3	La requête correcte : `curl -s -w ' -> %{http_code}\n' $U $H -d "$D" $URL`	`{"status":"accepted"} -> 200`.
T6.4	La même commande une seconde fois (rejeu).	`{"status":"duplicate"} -> 200`.
T6.5	**Rebond dur** de Karim : `curl -s -w ' -> %{http_code}\n' $U $H -d "$BOUNCE" $URL`	`200`.
T6.6	**Plainte** de Youssef : même commande avec `"$COMPLAINT"`.	`200`.
T6.7	**Réponse de Nadia** (SPF aligné sur son adresse) : même commande avec `"$REPLY"`.	`200`.
T6.8	**Expéditeur inconnu** : même commande avec `"$UNKNOWN"`.	`200`.
T6.9	**Usurpation** : une réponse « de Karim » dont le SPF est celui d'un **autre domaine** (`pirate@attacker.test`) : même commande avec `"$SPOOF"`.	`200` (l'événement est accepté, le tri se fait ensuite).

Attendre 15 secondes (le worker traite les événements), puis vérifier :

ID	VÉRIFICATION	RÉSULTAT ATTENDU
T6.10	`q "select c.email_normalized contact, m.status, c.do_not_contact from public.messages m join public.conversations v on v.organization_id=m.organization_id and v.id=m.conversation_id join public.contacts c on c.organization_id=v.organization_id and c.id=v.contact_id where m.provider_message_id like 'pm-guide-%' order by 1"`	**karim : bounced**, do_not_contact **f** ; **nadia : delivered**, f ; **youssef : accepted**, do_not_contact **t**.
T6.11	`q "select value_normalized, reason from public.suppressions order by 1"`	`karim@demo.test | bounce` et `youssef@demo.test | complaint`. **Un rebond dur supprime l'adresse mais ne touche pas « ne plus contacter »** (adresse invalide ≠ refus) ; **une plainte fait les deux**.
T6.12	`q "select c.email_normalized, m.direction, m.status, left(m.body,30) body from public.messages m join public.conversations v on v.organization_id=m.organization_id and v.id=m.conversation_id join public.contacts c on c.organization_id=v.organization_id and c.id=v.contact_id where m.provider_message_id = 'in-guide-1'"`	La réponse de Nadia : `inbound`, `received`, « Merci, je suis interessee. » **dans la conversation de Nadia**.

ID	VÉRIFICATION	RÉSULTAT ATTENDU
T6.13	`q "select from_normalized, reason, status from public.inbound_unmatched order by 1"`	`inconnu@exemple.test | unknown_sender | pending` et `karim@demo.test | sender_unverified | pending` : l'inconnu **et** l'usurpation sont dans la **file de tri**, pas dans la conversation de Karim.
T6.14	`qa "select count(*) from public.contacts where email_normalized like '%@exemple.test'"`	`0` : **aucun contact n'est créé** à partir d'un e-mail inconnu.
T6.15	`q "select external_event_id, status from public.webhook_events order by received_at"`	Les événements sont `processed` ; la ligne `rejected` (sans identifiant) est la trace du mauvais mot de passe de T6.1 : **sans aucun contenu**.

T7. Conversations et droits de lecture (J30)

Les conversations n'ont pas encore d'écran (boîte de réception : J37). Les vérifier en base, **comme chaque rôle** :

```
for u in 00000000-0000-4000-8000-000000000001 00000000-0000-4000-8000-000000000004 00000000-0000-4000-8000-000000000003; do docker exec -i
supabase_db_fusion-leap psql -U postgres -t -A <<SQL
begin; select set_config('request.jwt.claims', json_build_object('sub','$u','role','authenticated')::text, true); set local role
authenticated; select '$u ' || (select count(*) from public.conversations) || ' conversations, ' || (select count(*) from public.messages) ||
' messages'; commit;
SQL
done 2>&1 | grep conversations
```

ID	TEST	RÉSULTAT ATTENDU
T7.1	Résultat pour `…0001` (propriétaire d'Alpha).	**3 conversations, 7 messages** (une par contact, avec les messages sortants et la réponse de Nadia).
T7.2	Résultat pour `…0004` (commercial).	**0 conversation, 0 message** : le commercial ne lit que **ses** conversations (aucune ne lui est assignée).
T7.3	Résultat pour `…0003` (marketing).	**0 conversation, 0 message** : le marketing ne lit pas les conversations.
T7.4	La réponse de Nadia est dans une conversation à l'état **to_triage** : `qa "select state from public.conversations v join public.contacts c on c.id = v.contact_id where c.email_normalized = 'nadia@demo.test'"`	`to_triage` : la conversation n'a pas de responsable, elle attend le tri.

T8. Envoi sous contrôles (J31)

L'envoi d'un e-mail suit une chaîne de contrôles. Ce test **ne peut pas envoyer d'e-mail** : il prépare 3 messages, les fait **approuver** par le propriétaire (ce qui lance le job d'envoi) et vérifie que le worker **refuse** chaque envoi **avant** d'appeler le fournisseur.

```
docker exec -i supabase_db_fusion-leap psql -U postgres -v ON_ERROR_STOP=1 < docs/testing/fixtures/phase3-send-refusals.sql
```

Attendre 15 secondes, puis :

```
q "select c.email_normalized contact, m.status, m.error_code, m.provider_message_id from public.messages m join public.conversations v on
v.organization_id=m.organization_id and v.id=m.conversation_id join public.contacts c on c.organization_id=v.organization_id and
c.id=v.contact_id where m.idempotency_key like 'guide-refus-%' order by 1"
```

ID	CONTACT	RÉSULTAT ATTENDU	POURQUOI
T8.1	`nadia@demo.test`	`failed` / **`refused_no_consent`**	Aucun consentement enregistré pour cette finalité.
T8.2	`karim@demo.test`	`failed` / **`refused_suppressed`**	Son adresse est supprimée (rebond dur du test T6).

ID	CONTACT	RÉSULTAT ATTENDU	POURQUOI
T8.3	`youssef@demo.test`	`failed` / **`refused_do_not_contact`**	Il a porté plainte : « ne plus contacter ».
T8.4	Colonne `provider_message_id` des 3 lignes.	**Vide** : le fournisseur n'a **jamais** été appelé.	
T8.5	`q "select action from public.audit_logs where action in ('message.approved','message.failed') order by created_at desc limit 6"`	3 `message.approved` et 3 `message.failed` : l'approbation humaine et le refus sont tracés (jamais le texte du message).	

T9. Isolation et droits (vérifications rapides)

ID	COMMANDE	RÉSULTAT ATTENDU
T9.1	`pnpm test:call alpha-sales GET /api/integrations`	`HTTP 403`.
T9.2	`pnpm test:call beta-owner GET /api/integrations --org beta`	`HTTP 200` avec une liste **vide** : jamais les intégrations d'Alpha.
T9.3	`pnpm test:call alpha-analyst GET "/api/imports"`	`HTTP 403` : l'analyste n'a pas accès aux imports.

T10. Nettoyage

Arrêter le worker et l'application (Ctrl+C dans leurs terminaux), puis remettre la base à zéro avant de recommencer : `pnpm db:reset`.

Ce que ce guide ne couvre pas (à connaître)

- **Aucun e-mail réel n'est envoyé ni reçu** : le fournisseur n'a pas encore de compte d'essai, et aucun écran n'active un canal e-mail.
- **Désabonnement** (J33) : il n'existe pas encore ; aucun envoi marketing n'est possible avant.
- **Écran contacts et consentement** (J30b), **fiche lead** (J30c, J30d) et **boîte de réception** (J37) : pas encore construits ; les conversations se vérifient en base (T7).
- **WhatsApp** : pas encore construit.

Fiche d’anomalie

Pour chaque **KO**, noter : l'identifiant du cas (ex. B6), le compte utilisé, ce que vous avez fait, ce que vous avez vu, ce qui était attendu, la capture d'écran, et l'heure.

Fiche de résultats

Cocher **OK** ou **KO** pour chaque cas. En cas de KO, détailler dans la colonne **Commentaire** (ce qui a été vu) et joindre une capture d'écran. Testeur : _____ Date : _____ Version testée (commit) : ab54a68

A. Projets et produits

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
A1	Ouvrir Projets.			
A2	Changer d'organisation (Changer) pour Beta.			
A3	Nouveau projet : type Vente de sites web, nom Projet test 1, dans Offres et formules ajouter une…			
A4	Ouvrir Projet test 1.			
A5	Nouveau projet : laisser le nom vide, valider. Puis un nom de 121 caractères. Puis, dans une pag…			
A6	Dans Projet test 1, modifier une valeur (ex. ajouter Pack premium) et Enregistrer.			
A7	Section Produits et offres : Ajouter un produit : type pack, offre Offre de test, prix indicatif…			
A8	Ouvrir Modifier sur le produit, section Fichiers : envoyer une image PNG de moins de 5 Mo, puis …			
A9	Même endroit : envoyer un fichier .txt, puis un fichier de plus de 5 Mo, puis un fichier vide.			
A10	Cliquer Ouvrir (ou Télécharger) sur un fichier, puis Retirer.			
A11	Sur Projet test 1 : Archiver.			
A12	Ouvrir le projet archivé.			
A13	Avec alpha-sales@seed.test, alpha-marketing@seed.test puis alpha-analyst@seed.test : ouvrir Proj…			
A14	shared@seed.test → Administration → ouvrir Alpha → formule Essai, motif test quota, Enregistrer …			
A15	Rétablir la formule Croissance dans Administration, puis réessayer la création.			
A16	Nouveau projet : remplir, puis cliquer deux fois rapidement sur Créer le projet.			

B. Leads

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
B1	Ouvrir Leads.			
B2	Recherche nadia puis Appliquer. Puis xyz.			
B3	Réinitialiser. Choisir Étape = Unassigned puis Appliquer.			
B4	Réinitialiser. Cliquer le bouton Non assignés. Cliquer à nouveau (« Retirer le filtre « non assi…			
B5	Source = Import CSV, puis Formulaire web, puis Saisie manuelle.			
B6	Responsable = Membre 000004 (le commercial).			
B7	Réinitialiser, puis Tri = Nom (A–Z).			
B8	Projet = Application Salla Alpha, Appliquer.			
B9	Réinitialiser, puis Exporter en CSV. Ouvrir le fichier.			

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
B10	Vérifier l'audit de l'export : dans un terminal, q "select action, after from public.audit_logs ...			
B11	Avec alpha-sales@seed.test : ouvrir Leads.			
B12	Avec alpha-marketing@seed.test : ouvrir Leads. Avec alpha-analyst@seed.test : essayer d'ouvrir /...			

C. Deals

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
C1	Ouvrir Deals.			
C2	Sur la carte de Nadia, menu Déplacer vers... → Qualified.			
C3	Glisser-déposer la carte de Salma vers la colonne Won.			
C4	Déplacer le deal de Youssef (Won) vers New (réouverture), puis le remettre en Won.			
C5	Projet = Application Salla Alpha, Filtrer. Puis Tous les projets.			
C6	Avec alpha-sales@seed.test : ouvrir Deals, déplacer un deal.			
C7	Avec alpha-marketing@seed.test : ouvrir Deals.			
C8	Avec alpha-analyst@seed.test : essayer d'ouvrir /fr/deals.			

D. Intégrations

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
D1	shared@seed.test → Intégrations.			
D2	Vérifier que le jeton n'apparaît nulle part : clic droit → Afficher le code source de la page, c...			
D3	Section Remplacer le jeton : saisir court, Remplacer.			
D4	Saisir nouveau-jeton-0123456789, Remplacer.			
D5	Déconnecter.			
D6	Saisir un nouveau jeton et utiliser Reconnecter.			
D7	Avec alpha-admin@seed.test : ouvrir Intégrations. Avec alpha-marketing, alpha-sales et alpha-ana...			
D8	Avec beta-owner@seed.test : ouvrir Intégrations.			

T0. Suites de tests automatiques

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T0.1	pnpm lint && pnpm typecheck			
T0.2	pnpm test			
T0.3	pnpm test:db			
T0.4	pnpm test:rls			
T0.5	pnpm check:leaks			

T1. Capture publique de leads

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T1.1	La commande ci-dessus (origine autorisée).			
T1.2	La même commande une seconde fois (doublon).			
T1.3	Remplacer l'origine par -H "origin: https://pirate.test".			
T1.4	Remplacer la clé par flk_ suivi de 64 zéros.			
T1.5	Mettre "website":"robot" dans le corps (champ piège rempli).			
T1.6	Vérifier : q "select count(*) from public.contacts where email_normalized = 'rania@demo.test'"			
T1.7	Ouvrir Leads (shared@seed.test).			
T1.8	Vérifier les alertes : q "select count(*) from public.notifications"			

T2. Assignation automatique

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T2.1	Les 3 envois.			
T2.2	Après 10 secondes : q "select c.email_normalized, u.email as responsable from public.leads l joi...			
T2.3	Leads avec alpha-sales@seed.test.			

T3. Import CSV de leads

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T3.1	Envoi du fichier.			
T3.2	Aperçu.			
T3.3	Confirmation : pnpm test:call shared POST /api/imports/confirm " {"batch_id":"$BATCH","mapping":$...			
T3.4	Après 15 secondes : pnpm test:call shared GET "/api/imports?id=$BATCH"			
T3.5	pnpm test:call shared GET "/api/imports/report?batch_id=$BATCH"			
T3.6	Leads (shared@seed.test).			
T3.7	Avec alpha-sales, tenter l'import : pnpm test:call alpha-sales POST "/api/imports? project_id=$P"...			

T4. Synchronisation CRM par fichier

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T4.1	Création de l'intégration, puis du mapping.			
T4.2	Lancement de la synchro.			
T4.3	Après 15 secondes : q "select email_normalized, name, external_id from public.contacts where ema...			
T4.4	CRM (shared@seed.test).			
T4.5	Deals : la carte de Nadia.			

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T4.6	CRM avec alpha-sales, alpha-marketing, alpha-analyst.			

T5. Intégrations, secrets et secret de webhook

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T5.1	pnpm test:call alpha-sales POST /api/integrations/webhook-secret '{"integration_id":"$EID","secr…			
T5.2	Même commande avec shared et le secret court.			
T5.3	Même commande avec shared et le secret jeton-de-test-0123456789-abcdef (le même que le jeton d'e…			
T5.4	Même commande avec shared et mot-de-passe-de-webhook-0123456789-abcdef.			
T5.5	qa "select has_secret, has_webhook_secret, status from public.integrations where id = '$EID'"			
T5.6	qa "select count(*) from private.secrets where ciphertext like '%mot-de-passe%' or ciphertext li…			
T5.7	pnpm test:call beta-owner POST /api/integrations/webhook-secret '{"integration_id":"$EID","secre…			

T6. Webhooks e-mail : statuts et réponses

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T6.1	curl -s -o /dev/null -w '%{http_code}\n' -u demo-mail-guide:mauvais $H -d "$D" $URL			
T6.2	La même commande avec -u inconnu:mot-de-passe-de-webhook-0123456789-abcdef.			
T6.3	La requête correcte : curl -s -w ' -> %{http_code}\n' $U $H -d "$D" $URL			
T6.4	La même commande une seconde fois (rejeu).			
T6.5	Rebond dur de Karim : curl -s -w ' -> %{http_code}\n' $U $H -d "$BOUNCE" $URL			
T6.6	Plainte de Youssef : même commande avec "$COMPLAINT".			
T6.7	Réponse de Nadia (SPF aligné sur son adresse) : même commande avec "$REPLY".			
T6.8	Expéditeur inconnu : même commande avec "$UNKNOWN".			
T6.9	Usurpation : une réponse « de Karim » dont le SPF est celui d'un autre domaine (pirate@attacker.…			
T6.10	q "select c.email_normalized contact, m.status, c.do_not_contact from public.messages m join pub…			
T6.11	q "select value_normalized, reason from public.suppressions order by 1"			
T6.12	q "select c.email_normalized, m.direction, m.status, left(m.body,30) body from public.messages m…			
T6.13	q "select from_normalized, reason, status from public.inbound_unmatched order by 1"			
T6.14	qa "select count(*) from public.contacts where email_normalized like '%@exemple.test'"			
T6.15	q "select external_event_id, status from public.webhook_events order by received_at"			

T7. Conversations et droits de lecture

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T7.1	Résultat pour ...0001 (propriétaire d'Alpha).			
T7.2	Résultat pour ...0004 (commercial).			
T7.3	Résultat pour ...0003 (marketing).			
T7.4	La réponse de Nadia est dans une conversation à l'état to_triage : qa "select state from public....			

T8. Envoi sous contrôles

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T8.1	nadia@demo.test			
T8.2	karim@demo.test			
T8.3	youssef@demo.test			
T8.4	Colonne provider_message_id des 3 lignes.			
T8.5	q "select action from public.audit_logs where action in ('message.approved','message.failed') or...			

T9. Isolation et droits

CAS	ÉTAPE (DÉBUT)	OK	KO	COMMENTAIRE
T9.1	pnpm test:call alpha-sales GET /api/integrations			
T9.2	pnpm test:call beta-owner GET /api/integrations --org beta			
T9.3	pnpm test:call alpha-analyst GET "/api/imports"			